

Capstone I – Functional Test Report

Smart Distributed Acoustic Calibration System (SDACS)

Kyle Dick (ID: 7206287), David Toma (ID: 8823349), Amy Wentzell (ID: 8820998)

Winter 2026 – Level 10

Bachelor of Applied Engineering – Electronic Systems Engineering, Conestoga
College

EECE74125 - Capstone Project I

Professor Dr. So-Ra Chung, Professor Dr. Reda Fayek

March 26, 2026

Table of Contents

I. PURPOSE	2
II. TEST EQUIPMENT	3
III. TEST CONFIGURATION AND SETUP	4
1. Firmware Configuration.....	4
2. Hardware Connections: System Block Diagram	5
3. Server Setup.....	6
IV. TEST PROCEDURE	7
1. Network / NodeRED Verification	7
2. Data Logging	8
3. SD Card Logging (Local)	9
V. RESULTS	11
V. CHECKLIST.....	11

I. PURPOSE

The purpose of this test plan is to outline the functional testing of the Smart Distributed Acoustic Calibration System (SDACS). The functional testing for SDACS can be separated into the following functional units:

1. Network / NodeRED Compatibility
2. Data Logging (Audio, Temperature and Humidity)
3. SD Card Logging (Local)

II. TEST EQUIPMENT

Table 1: Test Equipment

Item No.	Equipment	Model / Part	Purpose
1	ESP32-S3 Development Board	Adafruit Metro ESP32-S3	Microcontroller used to capture digital microphone data and run signal processing firmware
2	Digital I2S MEMS Microphone	ICS-43434	Acoustic sensor used to capture sound pressure signals in digital I2S format
3	Server Host Computer	Raspberry Pi 5	The R-Pi 5 hosts the network server for NodeRED.
4	Test Signal Source	1 kHz (94 dB) Tone / Pink Noise Generator	Provides controlled acoustic test signals for calibration and validation
5	Sound Level Meter (Class 2)	Extech 407730	Reference instrument used to measure calibrated sound pressure level (SPL)
6	Loudspeaker / Studio Monitor	Reference Studio Monitor	Reproduces calibration tones at controlled SPL
7	Breadboard and Jumper Wires	—	Used for temporary electrical connections during testing
8	Personal Computer	Kyle's PC Processor AMD Ryzen 9 6900HX with Radeon Graphics (3.30 GHz) Installed RAM16.0 GB (15.3 GB usable)	Used for firmware flashing, serial monitoring, and data logging
9	USB-C Cable	—	Provides power and communication between ESP32-S3 and PC
10	MicroSD Card	—	For Local Data Logging the Audio on the ESP32-S3 Development Board
11	MicroSD Card Reader	UGREEN Multi-function Card Reader Case	For reading the local data from the ESP32-S3 Development Board
12	Thermometer	—	For verifying the temperature readings on the ESP32-S3 Development Board

III. TEST CONFIGURATION AND SETUP

1. Firmware Configuration

- Repository: CapstoneI_SDACS
(https://github.com/AmyWentzel/CapstoneI_SDACS)
- Firmware Branch: feature/modular-test
- Sample rate: 48 kHz
- Bit depth: 24-bit
- I2S mode: RX (microphone input)
- Measurement window: 10–20 seconds

The selected sampling rate and digital audio configuration allow the ESP32-S3 to capture acoustic signals across the audible frequency range while maintaining sufficient dynamic range for acoustic analysis [4].

2. Hardware Connections: System Block Diagram

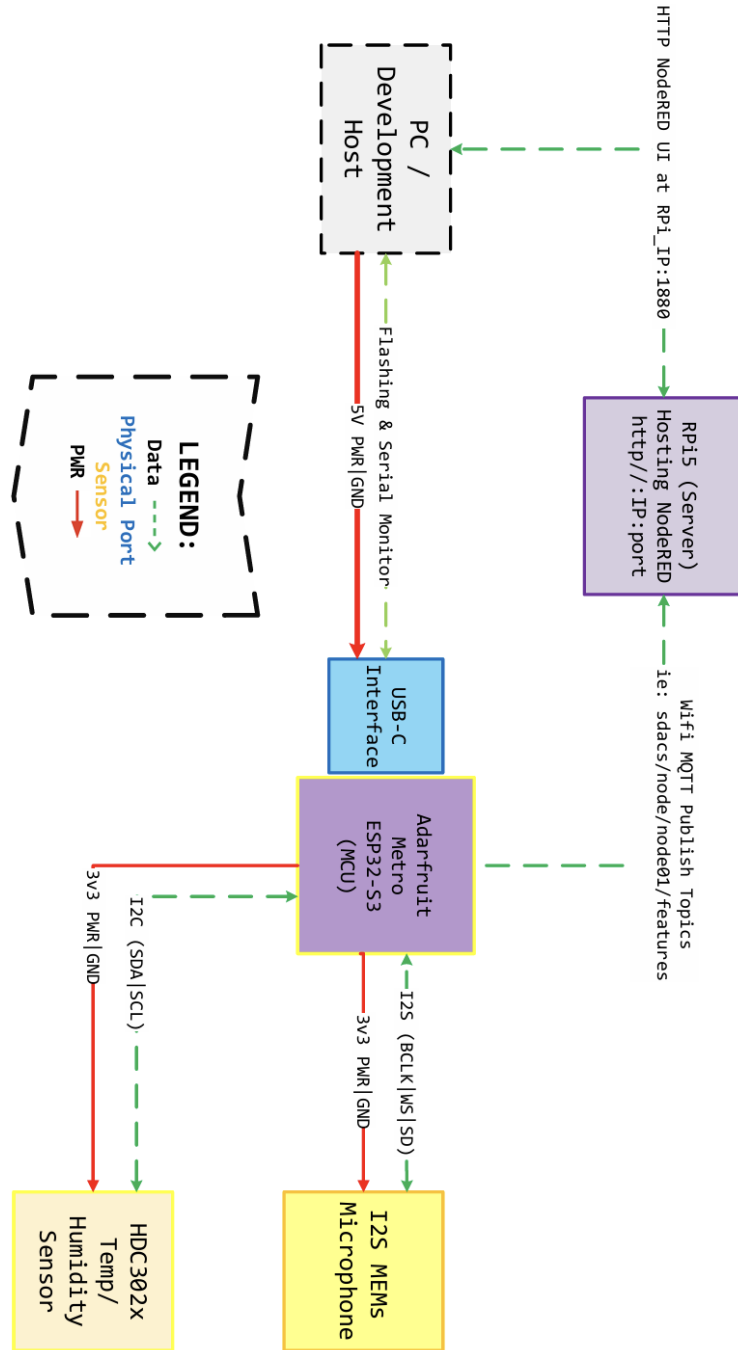


Figure 1: SDACS Connection Diagram

Table 2: Adafruit Metro Pinouts

ICS-43434 Pin	Connect To	ESP32-S3 Pin
VIN	Power	3.3V
GND	Ground	GND
BCLK	Bit Clock	GPIO14
LRCLK	Word Select	GPIO15
DOUT	I2S Data	GPIO16
SEL	Channel Select	GND

The ICS-43434 MEMS microphone outputs digital audio data using the I2S protocol, allowing direct interfacing with the ESP32-S3 without the need for analog amplification or external analog-to-digital conversion stages [3].

3. Server Setup

Ensure the server (R-Pi 5) and ESP32-S3 Development Board (Adafruit Metro) are configured to connect to the same local network. This can be done by modifying the Wi-Fi network SSID, IP, and Password in the file “sdacs_config.h”:

```

39  #define SDACS_PROVISION_WIFI_SSID      " "
40  #define SDACS_PROVISION_WIFI_PASS      " "
41  #define SDACS_PROVISION_MQTT_URI       "mqtt://172.20.10.5:1883"
42  #define SDACS_PROVISION_MQTT_TOPIC     "sdacs/node/node01/features"
43  #define SDACS_PROVISION_ALWAYS_SYNC_MQTT 1

```

Figure 2: Lines 39-43 of “sdacs_config.h”

IV. TEST PROCEDURE

1. Network / NodeRED Verification

- a) Measure and record the ambient temperature using the thermometer. Write the result in Celsius.
- b) Play the Test Signal Source on the Reference Studio Monitor. Measure and record the result of the sound pressure level (SPL) in decibels on the Sound Level Meter.
- c) From the GitHub branch “feature/modular-test”, run “app_main.c” on the Adafruit Metro S3¹.
- d) After the recording is complete and the code finishes building (~20 seconds), turn off the Reference Studio Monitor.
- e) The serial port terminal will also output the Wi-Fi / MQTT network setup results.

```
Build:Mar 27 2021
rst:0x15 (USB_UART_CHIP_RESET),boot:0x10
(DOWNLOAD(USB/UART0))
Saved PC:0x40041a76
--- 0x40041a76: ets_delay_us in ROM
waiting for download
--- Error: ClearCommError failed (PermissionError(13, 'The
device does not recognize the command.', None, 22))
--- Waiting for the device to reconnect.
I (415) heap_init: At 3FCF0000 len 00008000 (32 KiB): DRAI
(881) wifi:AP's beacon interval = 102400 us, DTIM period = 1
I (1791) esp_netif_handlers: sta ip: 172.20.10.2, mask:
255.255.255.240, gw: 172.20.10.1
I (1791) WIFI_MQTT: Got IP: 172.20.10.2
I (1791) WIFI_MQTT: WiFi IP: AP='Cheerios & Shreddies' RSSI=-
52 dBm CH=6 AUTH=WPA2_PSK
I (1801) WIFI_MQTT: MQTT client start.
Broker=mqtt://172.20.10.5:1883
I (1801) time_sync: Starting SNTP time sync...
I (1851) wifi:<ba-add>idx:0 (ifx:0, a2:45:14:bf:d0:12),
tid:0, ssn:2, winSize:64
I (1891) WIFI_MQTT: MQTT connected
I (3651) time_sync: SNTP synced time: 2026-03-16 19:15:01
```

Figure 3: Sample MQTT Setup Outputs on the ESP-IDF Terminal

- f) Verify that the IP matches the local network. Verify that MQTT is connecting at the start of the program (before recording / data logging starts).

¹ NOTE: The ESP-IDF extension on VS Code allows the user to

2. Data Logging

- g) The serial port terminal (e.g. ESP-IDF Terminal) will output both the audio measured in dB, and the temperature and humidity readings in Celsius and percentage per volume, respectively.

```
I (4001) run_storage: Created run folder:
/sdcard/node01_20260316_191501
I (4001) temp_humidity: Task started (period=2000ms,
addr=0x44)
I (4011) temp_humidity: Publishing temp/humidity to topic
'sdacs/node/node01/features/temp_humidity'
I (4021) audio_input: I2S initialized: BCLK=14 WS=15 DIN=16
SR=48000
I (4031) app_main: Capture started (20 s): MQTT stream + SD
logging
I (4031) capture_task: Waiting for WiFi+MQTT before
streaming...
I (4031) main_task: Returned from app_main()
I (4041) capture_task: WiFi+MQTT ready. Starting stream + SD
logging.
I (4041) temp_humidity: T=25.18 C, RH=25.9 % (n=1)
I (4051) temp_humidity: Temp/humidity start snapshot
published
I (5181) capture_task: LAeq=91.60 dB peak=94.00 dB
streamed=16384
I (6071) temp_humidity: T=25.20 C, RH=25.6 % (n=2)
```

Figure 4: Sample Audio (Purple) and Temperature and Humidity (Red) on the ESP-IDF Terminal

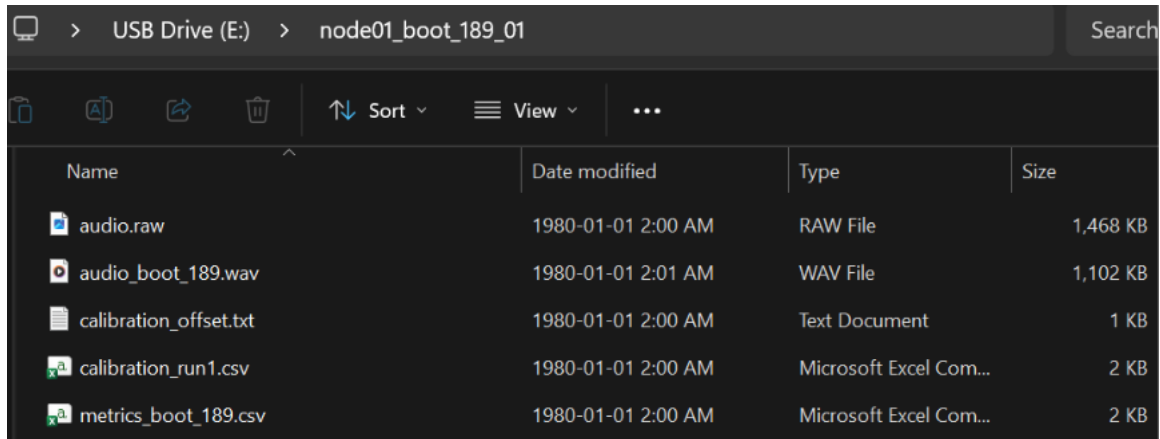
- h) Verify that audio readings reflect that the Adafruit Metro S3 board is recording properly (i.e readings are not zero).
- i) Verify that the decibels measured do not vary (test signals should be 94 dB). Accept ± 4 dB difference between measurements and the actual signal power in decibels.
- j) Verify that temperature readings are accurate by comparing the values from the serial port terminal to the thermometer. Accept $\pm 4^{\circ}\text{C}$ difference.

3. SD Card Logging (Local)

- k) The serial port terminal will output the SD Card detected on the ESP32-S3 Development Board.

```
Name: NCard
Type: SDHC/SDXC
Speed: 20.00 MHz (limit: 20.00 MHz)
Size: 14910MB
CSD: ver=2, sector_size=512, capacity=30535680 read_bl_len=9
SSR: bus_width=1
```

- l) Verify that the SD Card is successfully detected and identified.
 m) Remove the MicroSD card and connect it to the PC via MicroSD Card Reader.
 n) Verify that the folder structure follows the same format as below:



Name	Date modified	Type	Size
audio.raw	1980-01-01 2:00 AM	RAW File	1,468 KB
audio_boot_189.wav	1980-01-01 2:01 AM	WAV File	1,102 KB
calibration_offset.txt	1980-01-01 2:00 AM	Text Document	1 KB
calibration_run1.csv	1980-01-01 2:00 AM	Microsoft Excel Com...	2 KB
metrics_boot_189.csv	1980-01-01 2:00 AM	Microsoft Excel Com...	2 KB

Figure 5: Node-level File Structure on the MicroSD Card

- o) Open the file “calibration_run1.csv”. Verify that there is data being populated with proper timestamps:

	A	B	C	D	E	F	G	H	I	J
1	timestamp	node_id	LAeq_dB	peak_dB	dbfs	rms	temp_C	humidity	fft_peak_Hz	
2	2026-03-1'	node01	91.11	94	-2.89	0.716861	21.79	39.22	0	
3	2026-03-1'	node01	91.26	94	-2.74	0.729054	21.81	39.23	0	
4	2026-03-1'	node01	91.05	94	-2.95	0.711838	21.81	39.23	0	
5	2026-03-1'	node01	91.19	94	-2.81	0.723341	21.79	39.25	0	
6	2026-03-1'	node01	90.78	94	-3.22	0.690123	21.79	39.25	0	
7	2026-03-1'	node01	90.44	94	-3.56	0.663481	21.79	39.24	0	
8	2026-03-1'	node01	91.37	94	-2.63	0.738706	21.79	39.24	0	
9	2026-03-1'	node01	91.04	94	-2.96	0.711535	21.79	39.24	0	
10	2026-03-1'	node01	91.03	94	-2.97	0.710772	21.81	39.24	0	
11	2026-03-1'	node01	90.92	94	-3.08	0.701843	21.81	39.24	0	
12	2026-03-1'	node01	90.13	94	-3.87	0.640147	21.79	39.17	0	
13	2026-03-1'	node01	91.65	94	-2.35	0.763121	21.79	39.19	0	
14	2026-03-1'	node01	91.25	94	-2.75	0.728755	21.79	39.19	0	
15	2026-03-1'	node01	90.93	94	-3.07	0.702356	21.79	39.19	0	
16	2026-03-1'	node01	91.55	94	-2.45	0.753999	21.79	39.33	0	
17	2026-03-1'	node01	90.23	94	-3.77	0.64782	21.79	39.33	0	
18	2026-03-1'	node01	90.55	94	-3.45	0.672327	21.81	39.44	0	
19	2026-03-1'	node01	90.99	94	-3.01	0.707058	21.81	39.44	0	
20	2026-03-1'	node01	90.34	94	-3.66	0.655871	21.79	39.44	0	
21	2026-03-1'	node01	91.47	94	-2.53	0.747038	21.79	39.44	0	
22										

Figure 6: Node Audio Measurements in the CSV file (Local Data Logging)

V. RESULTS

Table 3: Temperature Reading Results

Thermometer Reading (°C)	Temperature Measured (°C)	Difference (°C)	Tolerance (%)	Pass/Fail
19.8	22.2	2.4	$\pm 4^{\circ}\text{C}$	Pass

Table 4: Sound Pressure Level (dB) Results

Test Signal Source (dB)	Sound Level Measured - Average(dB)	Difference (dB)	Tolerance (%)	Pass/Fail
93.7	96.9	3.2	$\pm 4\text{dB}$	Pass

V. CHECKLIST

Table 5: Conformance Criteria Checklist

Criteria	Pass / Fail
The ESP32-S3 Development Board recorded an accurate sound pressure level in decibels.	Pass
The ESP32-S3 Development Board recorded an accurate temperature measurement in Celsius.	Fail
The ESP32-S3 Development Board successfully connected to the Wi-Fi MQTT Server.	Pass
The ESP32-S3 Development Board successfully detected the SD Card onboard.	Pass
The ESP32-S3 Development Board successfully stored the raw audio data on the MicroSD Card with the appropriate file structure.	Pass
The file “calibration_run1.csv” contains non-zero audio data.	Pass

VI. OBSERVATIONS

1. Thermometer Reading Inaccuracy

The thermometer used for the temperature readings was a President's Choice meat thermometer. The difference in temperature reading between the thermometer and the temperature and humidity sensor was marginal but significant. It is believed that the discrepancy is caused by the likely lower accuracy of the reference thermometer, or heat generated by the development board.

2. Sound Level Meter Reading Variability

The Sound Level Meter used was a Class 2. The reading of the same audio source over time did vary quite a bit, requiring some estimation on the average SPL reading in decibels. Although the SPL readings on the development board passed the testing, it is recommended to perform the test again in a stricter lab environment.